

FAST National University of Computer and Emerging Sciences
Islamabad Campus

CityMind

An Urban Intelligence System
Final Implementation Report

Course: Artificial Intelligence
BS Computer Science — Group Project

Submitted By:

Laiba Amer (24i-0606)
Rida Zainab (24i-0521)
Nimra Zafar (24i0891)

Submission Date: 10th ay 2026

Table of Contents

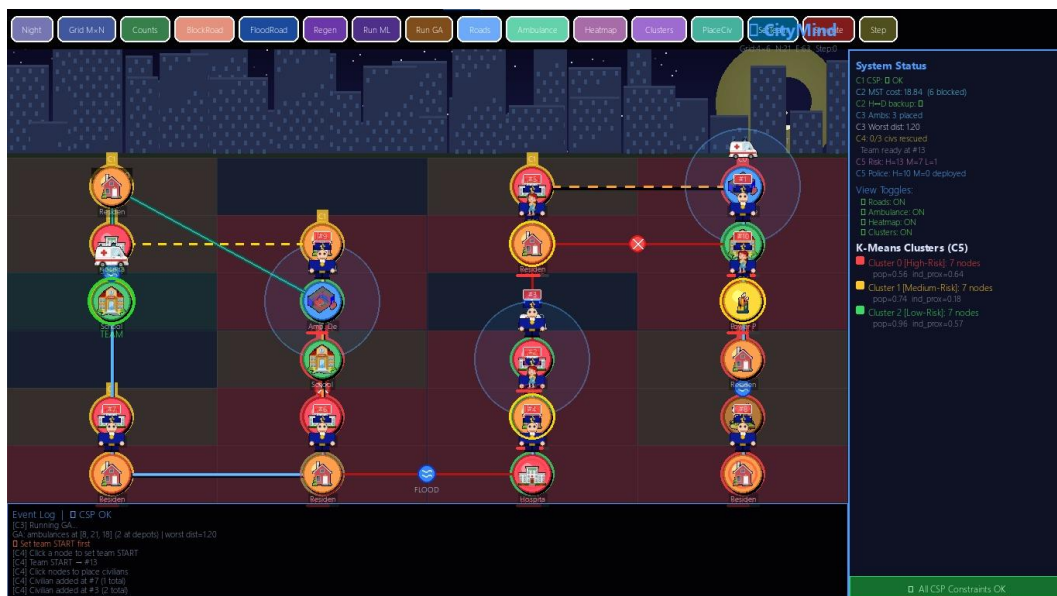
1. Project Overview
2. System Architecture & Shared Graph
3. Challenge 1 — City Layout Planning (CSP)
4. Challenge 2 — Road Network Optimization (MST)
5. Challenge 3 — Ambulance Placement (Genetic Algorithm)
6. Challenge 4 — Emergency Routing (A*)
7. Challenge 5 — Crime Risk Prediction (ML)
8. System Integration & 20-Step Simulation
9. Deviations from Design Document
10. Work Division
11. Conclusion

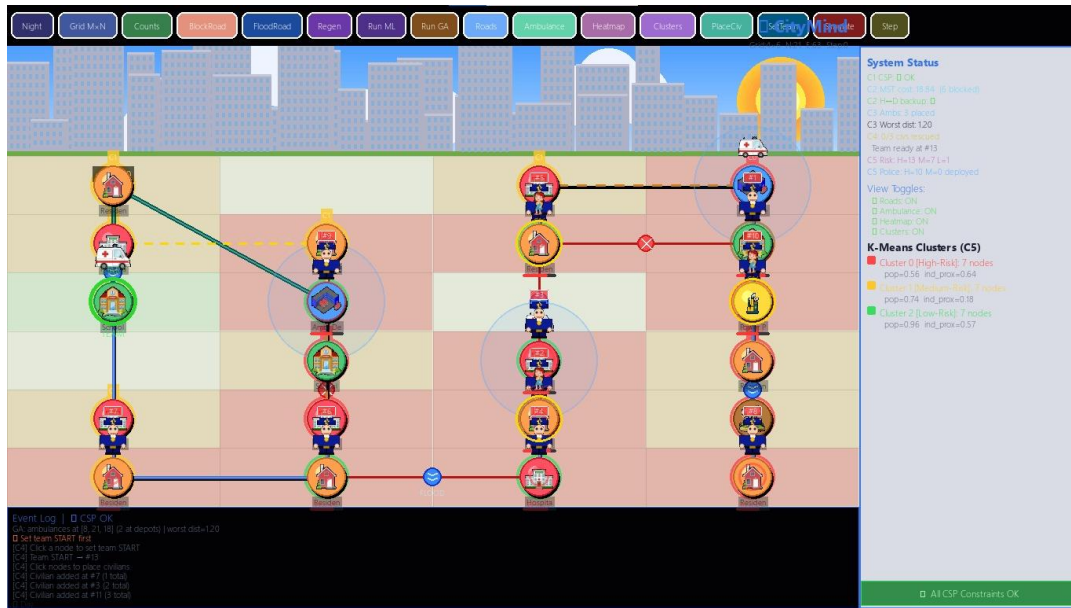
1. Project Overview

CityMind is a graph-based Artificial Intelligence system that models a mid-sized city as an interactive grid. The city is represented as a weighted graph where each node is a location (Residential, Hospital, School, Industrial, Power Plant, or Ambulance Depot) and each edge is a road with a travel cost. The system addresses five real-world operational challenges using five distinct AI techniques, all sharing a single city graph as the source of truth.

The goal of CityMind is to help city authorities make smarter, faster, and better-informed decisions about city layout, road infrastructure, ambulance deployment, emergency routing, and neighborhood risk assessment, all in real time.

UI Sketch (night time)



UI Sketch (Day time)**1.1 Technology Stack**

Component	Details
Language	Python 3.11
Graphics / UI	Pygame
External Libraries	None — all algorithms written from scratch
Grid Size	Configurable M×N (default 6×6, up to 16×14)
AI Techniques Used	CSP, Greedy MST, Genetic Algorithm, A* Search, K-Means + Decision Tree

2. System Architecture & Shared City Graph

The entire system is built around a single shared city graph implemented in graph.py. This graph is the architectural foundation of CityMind and acts as the single source of truth. No module is permitted to maintain its own private copy of city data. Any change made by one module — whether a road blockage, a risk level update, or a node type change — is instantly visible to all other modules.

2.1 Node Attributes

Each node in the graph stores the following information:

Attribute	Description
id	Unique integer identifier assigned automatically
type	One of: residential, hospital, school, industrial, powerplant, depot
population_density	Numeric value (Low=30, Medium=65, High=90)

risk_level	high / medium / low — assigned by ML module (Challenge 5)
risk_multiplier	1.5 (high) / 1.2 (medium) / 1.0 (low) — affects road costs
industrial_distance	BFS hops to nearest industrial zone — used by ML
accessibility	Boolean flag — whether the location is currently reachable
gx, gy	Grid coordinates (column, row)
px, py	Pixel coordinates on screen

2.2 Edge Attributes

Attribute	Description
base_cost	1.0 for standard roads, 0.8 for roads through residential zones
effective_cost	$\text{base_cost} \times \text{risk_multiplier}$ — used by A* and GA
blocked	True if road is manually blocked or flooded — skipped by all routing
flooded	True if blocked by flood event — shown with blue wave animation
in_mst	True if this edge is part of the Minimum Spanning Tree
redundant	True if this edge is a safety backup (not in MST but kept)

2.3 File Structure

File	Role
graph.py	Shared city graph — Node, Edge, CityGraph classes
csp.py	Challenge 1: Constraint Satisfaction Problem solver
mst.py	Challenge 2: Minimum Spanning Tree builder
ga.py	Challenge 3: Genetic Algorithm for ambulance placement
routing.py	Challenge 4: A* pathfinding and emergency routing
ml.py	Challenge 5: K-Means + Decision Tree crime risk prediction
ui.py	UI Manager — handles all button events, simulation loop, popups
renderer.py	Pygame renderer — draws grid, nodes, edges, overlays, animations
main.py	Entry point — initializes pygame and launches the application

3. Challenge 1 — City Layout Planning (CSP)

Challenge 1 asks: given a set of city locations on a grid, how should different building types be assigned so that important safety and zoning rules are never violated?

3.1 AI Technique: Constraint Satisfaction Problem (CSP)

We used a Constraint Satisfaction Problem solver with backtracking search (MRV + LCV heuristics) and AC-3 arc consistency. The solver guarantees that the initial city layout satisfies all three hard constraints before any node is placed on screen. If backtracking fails, an automatic self-healing repair function (`_auto_repair`) corrects any remaining violations.

3.3 Algorithm Details

- **MRV (Minimum Remaining Values):** The unassigned node with the fewest legal type options is assigned first, pruning the search space early.
- **LCV (Least Constraining Value):** Among legal types for a position, the one that eliminates the fewest options for neighbours is tried first.
- **AC-3:** Before backtracking, arc consistency reduces domains by removing type values that cannot possibly satisfy adjacency constraints.
- **Self-Healing:** After any runtime change (user changes a node type), `heal()` runs automatically. It repairs violations by swapping types of non-pinned nodes, leaving the user's chosen node fixed.

4. Challenge 2 — Road Network Optimization (MST)

Challenge 2 asks: given all the placed city nodes, how should roads be built to connect every location at minimum total cost, while also providing a backup path for resilience?

4.1 AI Technique: Greedy Minimum Spanning Tree (Kruskal's Algorithm)

We use a greedy Minimum Spanning Tree approach to build the road network. The MST connects all nodes with the minimum possible total road cost, ensuring no unnecessary roads are built. On top of the MST, the system also adds redundant safety edges, backup roads between Hospital and Depot nodes, to guarantee the city is still connected even if one road is blocked.

4.2 Road Cost Rules

- Standard road between any two nodes: cost = 1.0
- Road through a Residential zone: cost = 0.8 (lower, residential roads are prioritized)
- Effective cost = $\text{base_cost} \times \text{risk_multiplier of destination node}$ (set by ML module)
- Blocked roads are excluded from MST consideration entirely

4.3 Dynamic Rebuilding

Every time a road is blocked, a node type is changed, or the ML module updates risk weights, the MST automatically rebuilds using `rebuild_after_change()`. This ensures the road network is always optimal relative to the current graph state. The system also searches for two independent paths between Hospital and Depot, a primary (teal) and a backup (yellow dashed) and displays both in the UI.

5. Challenge 3 — Ambulance Placement (Genetic Algorithm)

Challenge 3 asks: given a city with residential zones (citizens) scattered across the grid, where should ambulances be stationed to minimize the maximum travel distance from any citizen to the nearest ambulance?

5.1 AI Technique: Genetic Algorithm (GA)

We implement a Genetic Algorithm from scratch. The GA evolves a population of candidate ambulance placements over multiple generations, using tournament selection, ordered crossover, and mutation to find the placement depot that minimizes the worst-case distance from any residential node to the nearest ambulance depot.

5.2 GA Parameters

Parameter	Value
Number of Ambulances	3
Population Size	50 chromosomes
Generations	100
Mutation Rate	10%
Elitism	Top 2 chromosomes pass unchanged to next generation
Selection	Tournament (size 3)
Fitness Function	Minimize worst-case Dijkstra distance from any citizen to nearest ambulance

5.3 Depot Snapping

After the GA identifies the 3 optimal node positions, the system snaps each ambulance to the nearest available Depot node. This ensures ambulances are visually stationed at their designated depot buildings rather than arbitrary nodes. Each ambulance is then shown as a sprite sitting at its depot, with a light-blue coverage circle drawn around it to indicate the zone it serves.

5.4 Dynamic Re-placement

Whenever the ML module updates risk weights (which change road effective costs), the GA automatically reruns and repositions ambulances to reflect the updated city conditions. This makes Challenge 3 fully dynamic.

6. Challenge 4 — Emergency Routing (A* Path finding)

Challenge 4 asks: given a medical team at a known starting position and a set of civilians at different nodes who need help, how should the team navigate the city to reach all civilians as efficiently as possible, even when roads are suddenly blocked by floods?

6.1 AI Technique: A* Search Algorithm

We implement A* search from scratch. A* finds the optimal path from the team's current position to the next civilian, using Manhattan distance as an admissible heuristic. The effective edge cost ($\text{base_cost} \times \text{risk_multiplier}$) is used as the path cost, so high-risk zones increase the cost of travelling through them. Blocked roads are automatically excluded from the search.

6.2 A* Heuristic

$h(n) = |gx_n - gx_{goal}| + |gy_n - gy_{goal}|$ (Manhattan distance on the grid). This heuristic is admissible because every grid move costs at least 0.8 (the minimum road cost), meaning the heuristic never overestimates.

6.3 Multi-Civilian Routing

The team visits civilians in the order they were placed. After reaching a civilian, A* immediately recalculates the route to the next one. The system supports any number of civilians. Each rescue is logged in the Event Log with step number and cost.

6.4 Real-Time Rerouting

Whenever a road is blocked, either manually by the user or automatically by a flood event during simulation, the system checks if that edge is on the current active path. If it is, A* reruns instantly from the team's current node position to the same target. The new path appears on screen immediately, demonstrating true dynamic rerouting.

6.5 Flood Events

A dedicated Flood Road button allows the user to manually trigger a flood on a random road at any time. During the simulation, floods also occur randomly (8% chance per step). Flooded roads are displayed with an animated blue wave marker, visually distinct from the red X used for manual road blocks.

6.6 Team Animation

The team ambulance smoothly slides between nodes during simulation rather than teleporting. This is achieved by linear pixel interpolation, each frame, the team's drawn position moves a fraction of the distance from the previous node to the next node, making movement visible and intuitive.

7. Challenge 5 — Crime Risk Prediction (Machine Learning)

Challenge 5 asks: can the city predict which neighbourhoods are at high, medium, or low risk of crime and use those predictions to automatically adjust how the rest of the system behaves?

7.1 ML Pipeline Overview

The ML module (ml.py) implements a two-stage machine learning pipeline entirely from scratch — no sklearn, no numpy, no external libraries. The two stages are:

- Step 1 (Unsupervised): K-Means Clustering with K-Means++ initialisation groups nodes into 3 clusters based on population density and industrial proximity. Clusters are labelled High, Medium, or Low risk by comparing centroid danger scores.
- Step 2 (Supervised): A Decision Tree (CART-style, max depth 4) is trained on synthetic data generated from the graph features. It predicts risk level for each node and for any new nodes added at runtime.

7.2 Features Used

Feature	Description
population_density (normalized)	Higher density → higher crime risk
industrial_distance (inverted, normalized)	Closer to industrial zone → higher crime risk. Distance computed via BFS.

7.3 Output & Integration

- Each node receives a risk_level (high/medium/low) and a risk_multiplier (1.5/1.2/1.0).
- graph.update_effective_costs() is called automatically, updating all edge costs.
- This immediately affects A* routing (high-risk zones cost more to traverse) and GA ambulance placement (which reruns with updated costs).
- The 10 highest-risk nodes receive police deployments, shown as blue star badges on the grid.

- A toggleable heatmap overlay colours each grid cell red (high), orange (medium), or green (low).

8. System Integration & 20-Step Simulation

All five challenges operate on the same shared CityGraph object. No module maintains its own copy of the data. The integration flow is as follows:

8.1 Startup Sequence

- Challenge 1 (CSP): Generates the initial city layout — all nodes placed according to constraints.
- Challenge 2 (MST): Builds the road network immediately after layout is generated.
- Challenge 5 (ML): User clicks 'Run ML', risk levels and multipliers written to all nodes. Edge costs update automatically.
- Challenge 3 (GA): User clicks 'Run GA', ambulances placed at optimal depot positions using updated costs.
- Challenge 4 (A*): User places civilians and sets team start. Clicking 'Simulate' begins the mission.

8.2 During Simulation (20 Steps)

- Each step advances the team one node along the A* path.
- At 8% probability per step, a random road is automatically flooded. The flood is visually marked and A* instantly reroutes if the team's path is affected.
- As risk weights may shift (user can rerun ML during simulation), GA automatically repositions ambulances and A* recalculates accordingly.
- The Event Log records every action: road blocked, reroute triggered, civilian reached, risk updated.

9. Deviations from Design Document (Deliverable 1)

Our implementation closely follows the design document submitted in Deliverable 1. The core algorithm choices, CSP with backtracking, Greedy MST, Genetic Algorithm, A*, and K-Means + Decision Tree remain exactly as designed. However, several enhancements were made during implementation that go beyond the original plan:

Design Document Plan	Actual Implementation & Justification
CSP runs once at startup	CSP also runs as a self-healing repair after every runtime node change. Justification: the project statement requires the system to prevent violations dynamically, not just at initialization.
Ambulances placed at optimal nodes (any type)	Ambulances are snapped to depot nodes after GA finds optimal positions. Justification: in a real city, ambulances must be stationed at designated depots, not arbitrary locations. This makes the visualization accurate and meaningful.
Road blocking only (manual)	Added Flood Road button for manual flood triggering, plus automatic random floods during simulation (8% per step). Justification: the project requires dynamic events; floods are more realistic than generic blocks and visually distinct.

Team movement (instant node-to-node)	Team movement is smoothly animated using linear pixel interpolation between nodes. Justification: the project requires showing simulation of movement; an animated team is far more readable than teleportation.
Grid: fixed size	Grid is fully configurable via 'Grid M×N' popup (3–16 columns, 3–14 rows). Justification: makes the system flexible for demo; the project states the graph must be large enough to make problems meaningful.
UI sketch: basic layout	Full pygame UI with night/day mode, animated sky, sprites, heatmap overlay, coverage zones, pulsing animations, event log. Justification: the project awards marks for creative interface design and extra features.

10. Work Division

The project was divided among three members based on the five challenges. Each member was responsible for designing, implementing, testing, and being able to explain their assigned modules.

Member	Roll No.	Responsibilities
Laiba Amer	24i-0606	- Challenge 1: City Layout Planning (csp.py) - Challenge 2: Road Network Optimization (mst.py) - Shared graph design (graph.py) — node/edge structure - Initial city generation pipeline
Rida Zainab	24i-0521	- Challenge 3: Ambulance Placement (ga.py) - Challenge 4: Emergency Routing (routing.py) - A* implementation and multi-civilian routing - Flood events and real-time rerouting logic
Nimra Zafar	24i-0891	- Challenge 5: Crime Risk Prediction (ml.py) - K-Means Clustering and Decision Tree from scratch - UI Manager & Renderer (ui.py, renderer.py) - Integration, simulation loop, event log, heatmap overlay

10.1 Collaboration Points

Although each member led their assigned modules, the following areas required full-group collaboration:

- Designing the shared graph schema (node and edge attributes all three members agreed on this before any code was written.
- System integration testing, ensuring CSP output feeds correctly into MST, GA, A*, and ML.
- The 20-step simulation, designed and tested together to verify all modules interact correctly.
- Debugging constraint violations and verifying CSP heal logic across different grid configurations.

11. Conclusion

CityMind successfully demonstrates a fully integrated urban AI system that applies five distinct AI techniques to five real-world city management challenges, all sharing a single dynamic graph as the foundation.

The system satisfies all core requirements of the project:

- All five challenges are implemented and working correctly in isolation.
- All modules share the same city graph, any change propagates instantly.
- The 20-step simulation integrates CSP, MST, GA, A*, and ML in a coherent live scenario.
- The visual interface shows the city grid with road network overlay, ambulance coverage zones, and crime risk heatmap.
- The event log records every system decision during simulation.

Beyond the minimum requirements, we added smooth team movement animation, configurable M×N grid sizes, a dedicated Flood Road button with animated visual markers, ambulance snapping to depot nodes, and a full day/night mode with animated sky, demonstrating both technical depth and interface creativity.